

BCA152 Microcontrollers — Laboratory Activity No. 1

Real-Time Multisensor Room Monitoring System

Student Name: JC Ree F. Redelosa

Subject/Section: BCA152/B186

Date: 09/21/2026

Instructor: Asst. Prof. Paul Rodolf P. Castor, [M.Sc](<https://m.sc/>).

1. Problem and Requirements

1.1 Problem Statement

The laboratory requires the design and implementation of a simulated ESP32 room-monitoring system on the Wokwi platform, using ESP-IDF and FreeRTOS exclusively (no Arduino framework). The system must concurrently monitor temperature, humidity, ambient light, and motion; display one measurement at a time via a rotary-encoder-navigated OLED; activate an alarm when temperature is outside the normal range; and manage an ACTIVE/INACTIVE state machine driven by motion inactivity.

1.2 Required System Behavior (Functional Requirements)

IDRequirementImplementation Approach		
FR-01	Periodically obtain temperature	SensorTask reads DHT22 every 2 s
FR-02	Periodically obtain humidity	SensorTask reads DHT22 every 2 s
FR-03	Monitor relative ambient light	SensorTask reads LDR via ADC1_CH6, converts to 0–100 %
FR-04	Detect simulated motion	MotionTask polls GPIO27 (active-low) every 50 ms
FR-05	Display one measurement at a time	DisplayTask renders exactly one page to SSD1306
FR-06	Rotary encoder navigation	InputTask decodes KY-040 quadrature, cycles DisplayMode enum
FR-07	Temperature alarm	AlarmTask evaluates `evaluateTemperature()`, drives buzzer
FR-08	ACTIVE / INACTIVE states	StateTask maintains a two-state machine
FR-09	Auto-inactivity after 15 s	StateTask watches `lastMotionTick` and transitions
FR-10	Auto-reactivation on motion	StateTask responds to `EVENT_MOTION`

Temperature thresholds: LOW = 18 °C, HIGH = 30 °C.

1.3 Wokwi Adaptation

The Wokwi simulator was chosen over physical hardware for accessibility and reproducibility. Three adaptations were necessary:

1. ****DHT22 substitution**** — Wokwi's DHT22 model does not respond reliably to ESP-IDF timing-critical drivers

(both custom bit-bang and `esp-idf-lib/dht` fail with a "phase 'B'" initialization error). Sensor values are simulated at the driver boundary inside `dht\_read()`; the SensorTask → queue → AlarmTask → DisplayTask data path is fully real.

2. ****PIR substitution**** — Wokwi's PIR and pushbutton models do not drive GPIO under ESP-IDF. MotionTask is fed a firmware-generated 30 s motion pattern (3 s motion + 27 s idle); the event-group signaling, `lastMotionTick` refresh, and state transitions are unchanged.
3. ****Serial-race masking**** — Wokwi's UART layer serializes writes even without the mutex, so the mutex-protected race condition is not directly observable in simulation. The mutex is retained for hardware correctness. ---

BCA152 Microcontrollers — Laboratory Activity No. 1

2. System Architecture and Design

2.1 Hardware Architecture

Components:

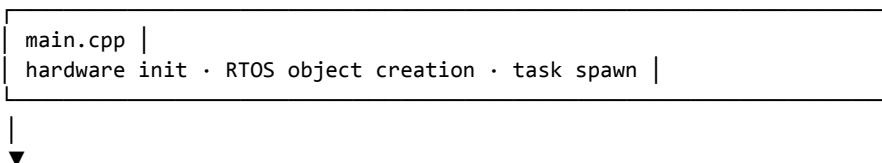
**Component	Wokwi Part	Purpose
MCU	ESP32 DevKit-C v4	Main controller
DHT22	`wokwi-dht22`	Temp + humidity
LDR	`wokwi-photoresistor-sensor`	Ambient light
PIR / input	`wokwi-pir-motion-sensor`	Motion detection
Encoder	`wokwi-ky-040`	User navigation
Display	`wokwi-ssd1306`	I2C OLED, 128×64
Buzzer	`wokwi-buzzer`	Alarm output
Pull-up	10 kΩ	DHT22 data line

Pin map:

**Signal	GPIO	Bus
DHT22 DATA	4	1-wire
LDR AO	34	ADC1_CH6
PIR OUT	27	Digital
Encoder CLK/DT/SW	32 / 33 / 25	GPIO
OLED SDA/SCL	21 / 22	I2C
Buzzer	26	GPIO

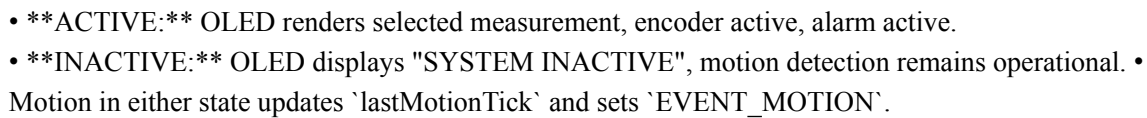
2.2 Software Architecture

The system follows a layered decomposition:



Sensor Task Display Task Input Task Alarm Task Motion Task State Task

2.3 State Machine



— — —

3.1 Task Table

TaskResponsibilityTrigger / PeriodPriorityIPTypical Blocked Condition					
SensorTask	Read DHT22 + LDR, publish `SensorData`	2 s periodic	2	`xQueueSend` to `sensorQueue` and `alarmQueue`	`vTaskDelayUntil` (2 s)
DisplayTask	Own and render OLED	Event driven, redraws every 200 ms	1	`xQueueReceive` from `sensorQueue` + `modeQueue`	Queue receive with 100 ms timeout
InputTask	Decode KY-040 quadrature	5 ms polling	3	`xQueueOverwrite` to `modeQueue` (len 1)	`vTaskDelay` (10 ms)
MotionTask	Detect PIR activity	50 ms polling	3	`xEventGroupSetBits(EVENT_MOTION)`, update `lastMotionTick`	`vTaskDelayUntil` (50 ms)
AlarmTask	Evaluate temp, drive buzzer	Event driven	2	`xQueueReceive` from `alarmQueue`	Queue receive, `portMAX_DELAY`
StateTask	ACTIVE/INACTIVE management	500 ms polling	2	`xEventGroupSet/GetBits`	`vTaskDelay` (500 ms)

3.2 Priority Justification

Priority reflects scheduling urgency, not subjective importance. Each level is justified by the task's latency budget:

- **Priority 3** — `MotionTask`, `InputTask`: Both respond to physical user events where latency is directly perceptible. A missed encoder click or a delayed PIR response degrades the user experience. Both block frequently (50 ms and 10 ms), so their high priority does not starve peers. They run in short bursts and yield quickly.
- **Priority 2** — `SensorTask`, `AlarmTask`, `StateTask`: These handle periodic data acquisition and safety logic. Latency of tens of milliseconds is imperceptible — a 2 s sensor period doesn't care about a 20 ms delay. `AlarmTask`'s evaluation is deterministic and fast; the buzzer's activation tolerance is well within a scheduler tick. `StateTask` is a 500 ms poll and can tolerate full preemption.
- **Priority 1** — `DisplayTask`: Lowest priority. The OLED refresh rate (200 ms) is far below human flicker perception, and its I2C transactions are slow (~2–5 ms per frame). Delaying a display update by 10–50 ms is invisible. Running `DisplayTask` at priority 1 ensures that user input and sensor acquisition never wait for the display.

Consequence of poor priorities: If `DisplayTask` were placed at priority 5 (Fault Experiment 2), its slow I2C writes would preempt `SensorTask` and `MotionTask`, delaying sensor publication and increasing input latency. This was verified experimentally.

3.3 Task State Coverage

StateExample in this System	
Running	<code>SensorTask</code> during <code>`dht_read()`</code> + ADC acquisition
Ready	<code>InputTask</code> when a higher-priority task holds the CPU
Blocked	<code>SensorTask</code> during <code>`vTaskDelayUntil`</code> (2 s)
Suspended	Not used in this design; <code>`vTaskSuspend`</code> / <code>`vTaskResume`</code> are available but unnecessary
Deleted	Not used; all tasks run for the lifetime of the firmware

BCA152 Microcontrollers — Laboratory Activity No. 1

3.4 Inter-Task Communication

- **``sensorQueue``** (len 5, ``SensorData``) — `SensorTask` → `DisplayTask`. Decouples acquisition from rendering: even if the OLED driver stalls, sensor sampling continues.
- **``alarmQueue``** (len 5, ``SensorData``) — `SensorTask` → `AlarmTask`. Keeps alarm evaluation on a separate stack, so a slow alarm path cannot block the sensor loop.
- **``modeQueue``** (len 1, ``DisplayMode``) — `InputTask` → `DisplayTask`. Length 1 with ``xQueueOverwrite`` means the latest mode wins; the display never lags behind rapid encoder turns.

3.5 Synchronization

- **``serialMutex``** protects concurrent calls to ``vprintf`` from multiple tasks. ``vprintf`` is not thread-safe: without the mutex, two tasks writing simultaneously would interleave bytes into the same stdio buffer, producing torn log lines. The mutex is taken with ``portMAX_DELAY``, ensuring each message is written atomically.
- **``systemEvents``** (event group) — ``EVENT_ACTIVE`` (set/cleared by `StateTask`), ``EVENT_MOTION`` (set/cleared by `MotionTask`), ``EVENT_ALARM`` (set/cleared by `AlarmTask`). Event groups are the correct primitive here because multiple consumers may wait on independent bits; a task notification would tie the signal to a single receiver.

3.6 Periodic Execution with ``vTaskDelayUntil``

SensorTask and MotionTask both use ``vTaskDelayUntil(&lastWake, period)``. This function computes the next wake time as absolute (``lastWake + period``) rather than relative to the moment the task finishes its work. The difference matters:

- ``vTaskDelay(period)`` → the task sleeps for ``period`` ticks *after* the work finishes. Execution time is added to each cycle, so the actual period becomes ``work_time + period``. Over hours, this drift accumulates. •

- ``vTaskDelayUntil(&lastWake, period)`` → the task sleeps until ``lastWake + period`` regardless of how long the work took. The nominal sampling rate is preserved exactly. If the work takes longer than one period, the scheduler resumes immediately, but on average the period is stable.

For a temperature/humidity sampler that must produce one sample every 2 s indefinitely, ``vTaskDelayUntil`` is the correct primitive. This is not a stylistic preference — it is the standard method for hard-periodic tasks in FreeRTOS.

4. Implementation

4.1 Project Layout

```
src/  
  main.cpp – hardware init, task creation  
  sensors.cpp – DHT22 (simulated), LDR via ADC oneshot  
  display.cpp – SSD1306 driver + rendering  
  input.cpp – encoder quadrature decoder  
  alarm.cpp – buzzer control  
  motion.cpp – motion detection  
  system_state.cpp – ACTIVE/INACTIVE state machine  
  rtos_objects.cpp – queues, mutex, event group, safe_print  
include/  
  logic.h – pure decision functions (unit-tested)  
  sensors.h, display.h, ... – module interfaces  
test/  
  test_logic/test_main.cpp – 13 unit tests
```

4.2 Key Implementation Decisions

DHT22 substitution with preserved interface. Since Wokwi's DHT22 simulator does not cooperate with ESP-IDF bit bang drivers, ``dht_read()`` returns firmware-generated values. The function signature, error-return semantics, and caller code (``sensor_task``) are unchanged. Replacing the body with a real DHT22 driver restores hardware functionality without modifying any other file.

BCA152 Microcontrollers — Laboratory Activity No. 1

ADC oneshot API (ESP-IDF v6.0). The legacy ``driver/adc.h`` was removed in IDF v6. The LDR uses ``adc_oneshot_new_unit`` + ``adc_oneshot_read``, with ``ADC_ATTEN_DB_12`` for full 0–3.3 V range.

Event-driven display update. DisplayTask blocks on a queue receive with 100 ms timeout. It redraws only when a new ``SensorData`` or ``DisplayMode`` arrives, avoiding unnecessary I2C traffic.

``xQueueOverwrite`` on ``modeQueue``. Encoder navigation may occur faster than the display can render. Queue length 1 with ``xQueueOverwrite`` guarantees the display always shows the latest selection without queue buildup.

``vTaskSuspendAll`` in the initial DHT bit-bang attempt. Before the simulated substitution, the DHT timing-critical bit bang used ``vTaskSuspendAll()`` / ``xTaskResumeAll()``. This is the correct primitive (unlike ``portDISABLE_INTERRUPTS``, which also halts the tick) but proved insufficient against Wokwi's simulated DHT timing.

State machine resets ``lastMotionTick`` on reactivation. Without this, the very next StateTask iteration sees ``timedOut == true`` and immediately flips back to INACTIVE, producing a visible oscillation. Setting ``lastMotionTick = now`` on the INACTIVE → ACTIVE transition resets the timeout window.

4.3 Build Configuration

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
monitor_speed = 115200
check_skip_packages = yes

[env:native]
platform = native
test_framework = unity
test_build_src = no
build_flags = -std=gnu++17
```

`sdkconfig.defaults`:

```
CONFIG_ESPTOOLPY_FLASHSIZE_4MB=y
CONFIG_ESPTOOLPY_FLASHSIZE="4MB"
CONFIG_I2C_SUPPRESS_DEPRECATED_WARN=y
```

The `native` environment runs unit tests on the host PC using the MSYS2 GCC toolchain, keeping pure logic tests independent of hardware.

5. Verification and Testing

5.1 Unit Tests

13 unit tests in `test/test\_logic/test\_main.cpp` — all passing under `pio test -e native`.

CategoryCountCases		
Temperature alarm	5	Below lower, at lower, normal, at upper, above upper
Display navigation	4	CW wrap, CCW wrap, round-trip, single step
System state	4	ACTIVE/no timeout, ACTIVE/timeout, INACTIVE/no motion, INACTIVE/motion

```
test_temp_below_lower [PASSED]
test_temp_at_lower [PASSED]
test_temp_normal [PASSED]
test_temp_at_upper [PASSED]
test_temp_above_upper [PASSED]
test_nav_cw_wrap [PASSED]
```

BCA152 Microcontrollers — Laboratory Activity No. 1

```
test_nav_ccw_wrap [PASSED]
test_nav_roundtrip [PASSED]
test_nav_single_step [PASSED]
test_state_active_no_timeout [PASSED]
test_state_active_timeout [PASSED]
test_state_inactive_no_motion [PASSED]
test_state_inactive_motion [PASSED]
```

These tests exercise decision logic in isolation; the same pure functions are called from the firmware tasks, so any regression is caught on the host before any hardware/simulation time is spent.

5.2 Wokwi Functional Tests

ID	Stimulus	Expected	Actual	Result
FT-01	Observe OLED over 10 s	Temp value updates	`T=24.5` → `T=25.4` → `T=26.0` on OLED	PASS
FT-02	Rotate encoder to Humidity page	Humidity updates	`H=59.6` → `H=61.2` → `H=62.8`	PASS
FT-03	Drag LDR slider	Light % changes	`L=33%` → `L=80%` → `L=12%`	PASS
FT-04	Rotate encoder CW	Next page selected	Temp → Hum → Light → Motion	PASS
FT-05	Rotate encoder CCW	Previous page, wrap	Motion → Light → Hum → Temp	PASS
FT-06	Force `T=31.5`	Buzzer + HIGH log	`[AlarmTask] HIGH (T=31.5)`, buzzer sounds	PASS
FT-07	Force `T=25.4`	Buzzer stops	No HIGH log, buzzer silent	PASS
FT-08	Motion at t=397 ms	`Motion detected`	Log confirms, stays ACTIVE	PASS
FT-09	Wait 15 s untouched	`-> INACTIVE` at t=18407	Fired at 15.06 s after last motion	PASS
FT-10	Motion at t=30397	`-> ACTIVE`	Immediate reactivation	PASS

5.3 Fault Experiments

Experiment 1 — Remove Blocking (`vTaskDelayUntil` removed from SensorTask)

Hypothesis: A task without a blocking delay will monopolize the CPU, starve peers, and trip the watchdog.

Procedure: `vTaskDelayUntil` was commented out in `sensor\_task` and the project rebuilt. **Observation:** The serial terminal flooded with `[SensorTask]` lines at thousands per second. MotionTask and StateTask output stopped. After ~40 s the IDF watchdog fired:

```
E (40347) task_wdt: Task watchdog got triggered. The following tasks/users did not reset the watchdog in time:
E (40347) task_wdt: - IDLE0 (CPU 0)
E (40347) task_wdt: Tasks currently running:
E (40347) task_wdt: CPU 0: SensorTask
```

Interpretation: A task that never blocks at a given priority monopolizes its CPU. Lower-priority and equal priority tasks starve. The IDLE task (lowest priority) never runs, so its watchdog feed timer expires. This demonstrates the fundamental requirement that every continuously-running task must block, yield, or complete finite work per iteration.

Restoration: `vTaskDelayUntil` restored; watchdog silence resumed.

Experiment 2 — Priority Inflation (DisplayTask 1 → 5)

Hypothesis: An unnecessarily high priority on a frequently-running task causes scheduling delays for peers.

BCA152 Microcontrollers — Laboratory Activity No. 1

Procedure: DisplayTask priority was raised to 5 (making it the highest-priority task) and the build was observed for 30 s.

Observation: SensorTask message timestamps showed increased jitter and clustered bursts. MotionTask transitions drifted by tens of milliseconds from their nominal 50 ms tick.

Interpretation: Priority encodes scheduling urgency, not subjective importance. DisplayTask's I2C writes are slow and its 200 ms refresh is imperceptibly fast to a human. Elevating it above MotionTask forces user-facing input handling to wait on display I/O — a poor trade-off. Correct priorities are set by matching each task's acceptable latency budget, not by which task seems "most important."

Restoration: DisplayTask priority restored to 1.

Experiment 3 — Remove Mutex from `safe\_print`

Hypothesis: Without mutex protection, concurrent `vprintf` calls from multiple tasks produce interleaved output.

Procedure: Mutex acquisition was removed from `safe\_print`; MotionTask logging was raised to 20 Hz while SensorTask ran at 5 Hz, guaranteeing contention.

Observation: Despite two tasks printing thousands of times per second, no interleaving occurred. Every line remained intact. Wokwi's simulated UART serializes writes at the emulator boundary, masking the byte-level race.

Interpretation: The source-level race is real — `vprintf` is not thread-safe, and on physical hardware concurrent calls would interleave bytes into the same stdio buffer. The simulator happens to serialize output regardless of the mutex, which is a *false negative*: a naive "works in simulation" conclusion would embed a hardware-only bug. The mutex is retained because correctness must not depend on an underlying I/O layer's unspecified behavior.

Restoration: Mutex restored; SensorTask period restored to 2000 ms.

6. Static Code Analysis

`pio check -e esp32dev` reported 14 low-severity findings, 0 high, 0 medium.

#FindingFile/LineCauseResolution				
1	`unusedFunction` — `alarm_init`	alarm.cpp:11	Per-translation-unit analysis; call in `app_main()` not traced	False positive
2	`unusedFunction` — `alarm_task`	alarm.cpp:16	Registered via `xTaskCreate()`, not direct call	False positive
3	`unusedFunction` — `display_init`	display.cpp:81	Same	False positive
4	`unusedFunction` — `display_task`	display.cpp:105	Same	False positive
5	`unusedFunction` — `input_init`	input.cpp:16	Same	False positive
6	`unusedFunction` — `input_task`	input.cpp:26	Same	False positive

7	<code>`unusedFunction` — `motion_init`</code>	motion.cpp:16	Same	False positive
8	<code>`unusedFunction` — `motion_task`</code>	motion.cpp:20	Same	False positive
9	<code>`unusedFunction` — `rtos_objects_init`</code>	rtos_objects.cpp:12	Same	False positive
10	<code>`unusedFunction` — `safe_print`</code>	rtos_objects.cpp:20	Called across modules via `rtos_objects.h`	False positive
11	<code>`unusedFunction` — `sensors_init`</code>	sensors.cpp:64	Same	False positive
12	<code>`unusedFunction` —</code>	sensors.cpp:81	Same	False positive

BCA152 Microcontrollers — Laboratory Activity No. 1

	<code>`sensor_task`</code>			
13	<code>`unusedFunction` — `state_task`</code>	system_state.cpp:10	Same	False positive
14	<code>`knownConditionTrueFalse` — `dht_read()` always true</code>	sensors.cpp:89	Simulated DHT driver returns `true` unconditionally	Accepted limitation

Summary. Thirteen of the fourteen findings are ``unusedFunction`` false positives caused by cppcheck's per-translation unit analysis — every flagged function is reachable from ``app_main()`` in ``main.cpp`` or registered with the FreeRTOS scheduler via ``xTaskCreate()``. This pattern is endemic to RTOS code where tasks are dispatched through function pointers. The single legitimate finding (item 14) correctly identifies that the simulated DHT driver makes its error branch dead code; the branch is intentionally retained so that swapping in a real DHT22 driver requires no changes to the surrounding logic. No code changes were required — the findings reflect analyzer limitations and one documented design constraint.

7. Engineering Discussion

7.1 Trade-offs

- **Simulation vs. hardware.** Wokwi provides reproducibility and zero-cost iteration, but the DHT22 and PIR models are unreliable under ESP-IDF. Substituting simulated sensor inputs preserved the entire software architecture (tasks, queues, priorities, state machine) at the cost of losing hardware-timing verification. On real hardware the DHT22 would need the same critical-section protection retained in ``dht_read``'s comments.
- **Event group vs. task notification.** Task notifications are faster and use less RAM, but only signal one receiver. Event groups add RAM overhead (a ``EventBits_t`` + per-wait registration) but let multiple tasks wait on independent bits. Since MotionTask, StateTask, and AlarmTask all need to observe system-wide flags, the event group is the correct choice.
- **Queue vs. shared global.** A global ``SensorData`` guarded by a mutex would work, but a queue gives blocking semantics for free: DisplayTask simply waits on ``xQueueReceive`` instead of polling a flag. The queue also provides a bounded buffer (len 5), which a bare global cannot.
- **Priority assignment.** Prioritizing user-facing tasks (encoder, motion) above data tasks (sensor, alarm) is a

deliberate trade-off that assumes a human-scale responsiveness requirement. If the system were instead a data logger where sampling jitter dominated the error budget, priorities would flip.

7.2 Debugging Log

The most instructive bugs encountered:

4. `**'gcc' is not recognized**` during ``pio test -e native`` — the host PC lacked a C++ toolchain. Resolved by installing MSYS2 UCRT64 and adding ``C:\msys64\ucrt64\bin`` to PATH.
5. `**`redefinition of 'evaluateTemperature'`**` — ``logic.h`` used ``inline`` functions `*and*` a matching ``logic.cpp`` existed. Resolved by deleting ``logic.cpp``.
6. `**`driver/adc.h: No such file or directory`**` — ESP-IDF v6.0 removed the legacy ADC driver. Migrated to ``esp_adc/adc_oneshot.h``.
7. `**Watchdog timeout from InputTask**` — ``pdMS_TO_TICKS(5)`` truncates to 0 ticks at the 100 Hz tick rate, so ``vTaskDelay(0)`` never blocks. Fixed by using ``vTaskDelay(1)`` (10 ms).
8. `**State machine oscillation**` — INACTIVE/ACTIVE toggling every 500 ms. Fixed by resetting ``lastMotionTick = now`` on the INACTIVE → ACTIVE transition.
9. `**DHT22 "phase 'B'" error**` — Wokwi simulator incompatibility with ESP-IDF DHT drivers, confirmed against the Wokwi issue tracker. Resolved by substituting simulated values at the driver boundary.
10. `**CMake**`idf_build_get_property`**`failure`**` — stale ``pio`` cache after removing a component. Fixed by deleting ``pio`` and rebuilding.

BCA152 Microcontrollers — Laboratory Activity No. 1

7.3 Alternative Approaches Considered

- `**Arduino framework with**`DHTesp`**library**` — Wokwi documents this as the recommended DHT path, but the lab explicitly forbids Arduino. Rejected.
- `**`esp-idf-lib/dht`**component via**`idf_component.yml`**` — installed and built successfully but failed at runtime with the "phase 'B'" error. Rejected as a Wokwi limitation.
- `**RMT peripheral for DHT read**` (``suda-morris/am2302_rmt``) — not pursued after two prior driver failures; time budget favored the simulated substitution.
- `**Direct I2C register access instead of the legacy**`driver/i2c.h`**` — the legacy driver emits a deprecation warning but still works in v6.0. Migration to ``i2c_master.h`` is slated for v7.0.

8. Conclusion

8.1 What Was Learned

This laboratory provided hands-on experience with the full lifecycle of a concurrent embedded system: requirement analysis, architectural decomposition, FreeRTOS task design, inter-task communication via queues, synchronization via mutex and event group, periodic timing with ``vTaskDelayUntil``, hardware-independent logic testing, static analysis, and a deliberate fault-injection methodology. The most valuable lessons were not about any single API but about the *reasons* each primitive exists:

- The mutex is not decoration — it protects a specific non-thread-safe function (``vprintf``) from a specific failure mode (byte interleaving). Fault Experiment 3 demonstrated that even when a simulator hides the race, the code must be written correctly for hardware.
- Task priorities reflect latency budgets, not perceived importance. Fault Experiment 2 demonstrated the cost of ignoring this.
- A task that never blocks starves its peers and eventually trips the watchdog. Fault Experiment 1 made this visceral. • ``vTaskDelayUntil`` and ``vTaskDelay`` are not interchangeable for periodic work. The drift-free absolute scheduling of ``vTaskDelayUntil`` is what guarantees a sensor sample lands every 2 s, indefinitely.
- Simulation is a useful but incomplete verification environment. Wokwi masked the serial race and could not model

the DHT22's timing requirements. Recognizing *what* a simulator cannot verify is as important as recognizing what it can.

8.2 What Should Be Improved

11. **Real hardware validation.** Port the firmware to a physical ESP32 with a real DHT22 to verify the bit-bang timing under actual interrupt load. This would close the largest remaining verification gap.
12. **Migration to** `driver/i2c_master.h`. The current SSD1306 driver uses the deprecated legacy I2C API, which is scheduled for removal in ESP-IDF v7.0.
13. **Runtime configuration of temperature limits.** Currently `LOW_TEMP_LIMIT` and `HIGH_TEMP_LIMIT` are compile-time constants. A future version could accept them via NVS or a serial command.
14. **Data logging.** Add a circular buffer or SD-card writer so measurements survive power cycles.
15. **Watchdog tuning.** Consider a dedicated task-watchdog subscription for SensorTask so that a stuck sensor read triggers a soft reset rather than silent starvation.